

AI FAQ ASSISTANT API

Project Documentation

A RESTful AI-Powered Frequently Asked Questions Management System

Project Details

Project Title: AI FAQ Assistant API

Team ID: SWTID-2026-6721

Team Size: 04

Team Leader: Bhuvaneswari . R

Email ID: 24bsccs22@cttewc.edu.in

Team Members

S. No.	Team Member Name	Email ID
1	Kaviya S	24bsccs56@cttewc.edu.in
2	Prathipa S	24bsccs03@cttewc.edu.in
3	Dharsini S	24bsccs54@cttewc.edu.in

Description

The **AI FAQ Assistant API** is a secure RESTful backend application designed to manage frequently asked questions and provide AI-powered answers. The application allows users to register and log in securely, manage FAQs, search existing FAQs, and generate answers using **Google Gemini AI**.

The project is developed using **Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt, and Google Gemini SDK (@google/genai)**. The application follows a modular **MVC architecture** with separate models, controllers, routes, middleware, and services.

Scenario-Based Case Study

Background

A customer support team receives many repeated questions from users. Manually creating, updating, searching, and answering these questions can be time-consuming.

Problem

- Repeated customer questions require manual answers.
- FAQs are difficult to organize without proper categorization.
- Users need secure authentication before modifying FAQs.
- Searching through a large number of FAQs can take time.
- Creating FAQ questions and answers manually is inefficient.

Solution

The **AI FAQ Assistant API** provides a centralized backend system for authentication, FAQ management, searching, and AI-powered answer generation.

Users can securely register and log in, create and manage their own FAQs, search available FAQs, and use Google Gemini AI to generate answers and FAQ question-answer pairs.

Usage

Authenticated User

- Register an account.

- Log in and receive a JWT token.
- View personal profile information.
- Create FAQs.
- Update and delete owned FAQs.
- Generate AI answers.
- Generate AI-based FAQ question-answer pairs.

Public User

- View available FAQs.
- Search FAQs using keywords.

Outcome

The system reduces manual FAQ management and provides a secure and organized platform for handling frequently asked questions. JWT authentication protects private operations, MongoDB stores application data, and Gemini AI provides automated answers.

1. Software Requirements

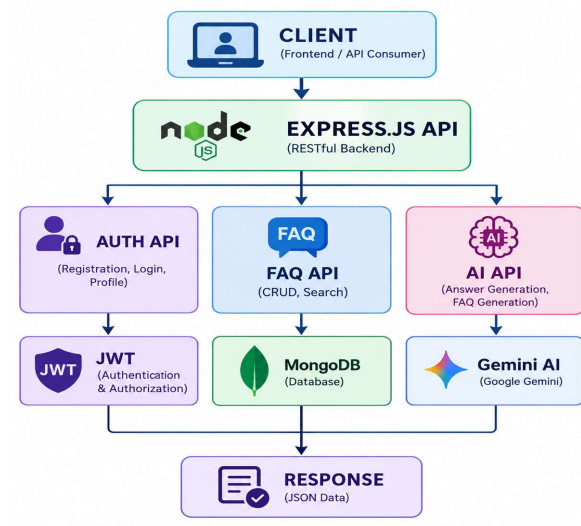
- **Operating System:** Windows 10/11, macOS, or Linux
 - **Node.js:** Version 18 or above
 - **npm:** Node Package Manager
 - **Express.js:** Backend REST API framework
 - **MongoDB:** Database
 - **Mongoose:** MongoDB ODM
 - **Postman / Thunder Client:** API testing
 - **Visual Studio Code:** Development environment
 - **Google Gemini API:** AI answer and FAQ generation
-

2. Hardware Requirements

- **Processor:** Intel Core i3/i5 or equivalent
 - **RAM:** Minimum 4 GB; 8 GB recommended
 - **Storage:** At least 1 GB available space
 - **Internet:** Required for Google Gemini API communication
-

3. System Architecture

The application follows a modular backend architecture



Components

- **Express.js API:** Handles incoming HTTP requests.
- **Authentication:** Uses JWT for protected routes.
- **MongoDB:** Stores users and FAQ records.
- **Mongoose:** Defines schemas and communicates with MongoDB.
- **Gemini AI:** Generates answers and FAQ content.
- **Middleware:** Handles authentication, validation, and errors.

4. Database Design

The application primarily contains two MongoDB entities.

User Entity

- `_id`
- `name`
- `email`
- `password`
- `role`
- `createdAt`
- `updatedAt`

Passwords are hashed using **bcrypt** before being stored.

FAQ Entity

- `_id`
- `question`
- `answer`
- `category`
- `createdBy`
- `createdAt`
- `updatedAt`

The `createdBy` field connects an FAQ with the user who created it.

5. Key Features

- **User Authentication**

JWT-based registration, login, and protected profile access.

- **Password Security**

User passwords are securely hashed using bcrypt.

- **FAQ Management**

Users can create, view, update, and delete FAQ records.

- **FAQ Search**

Users can search FAQs using keywords.

- **AI Answer Generation**

Google Gemini generates answers for user questions.

- **AI FAQ Generation**

Gemini generates a structured FAQ question and answer based on a given topic.

Category Management

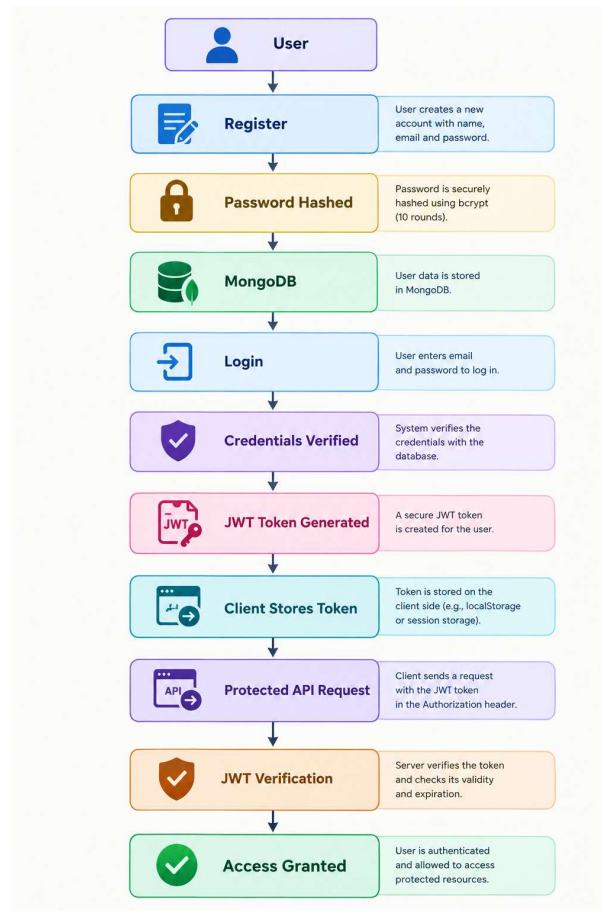
FAQs are organized using predefined categories such as:

- Technology
- Education
- Health
- Banking
- General

Centralized Error Handling

The application provides standardized error responses for invalid requests and server/database errors.

6. User Flow



7. MVC Architecture

The project follows the **Model-View-Controller** architecture.

- **Model**

The Model layer manages MongoDB schemas and data structures.

Examples:

`src/models/User.js`

`src/models/FAQ.js`

- **Controller**

The Controller layer processes requests and connects routes with business logic.

Examples:

`src/controllers/authController.js`

`src/controllers/faqController.js`

`src/controllers/aiController.js`

- **Routes**

Routes define the API endpoints available to the client.

`src/routes/authRoutes.js`

`src/routes/faqRoutes.js`

`src/routes/aiRoutes.js`

- **Services**

The Gemini service communicates with Google Gemini AI.

`src/services/geminiService.js`

- **Middleware**

Middleware handles authentication, validation, and errors.

`src/middleware/authMiddleware.js`

`src/middleware/validationMiddleware.js`

`src/middleware/errorMiddleware.js`

7. Project Setup

Creating the Project

Create the project folder and open it in Visual Studio Code.

`npm init -y`

Install the required dependencies:

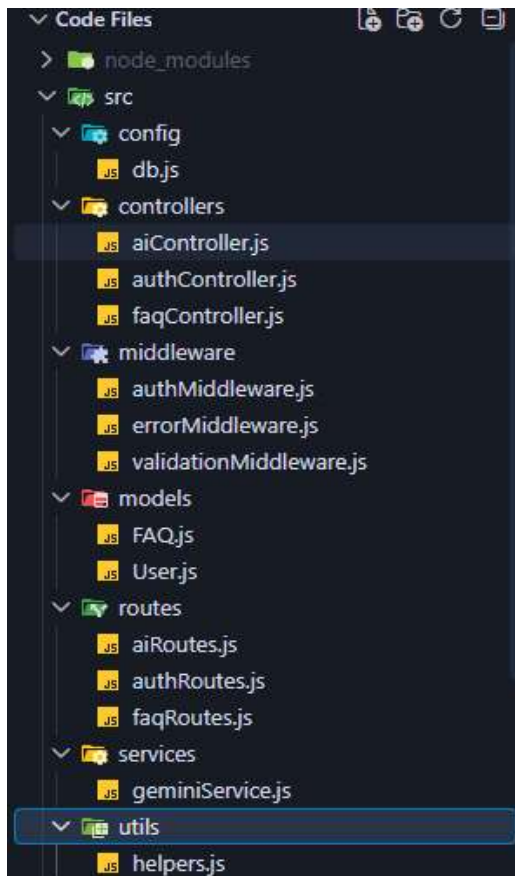
Clone or copy the project files to your directory and run:

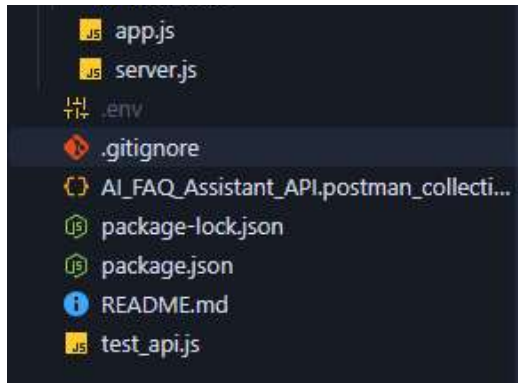
```
npm install
```

```
npm install express mongoose dotenv cors jsonwebtoken bcryptjs @google/genai
```

8. Project Architecture

The project follows a structure similar to the **MVC architecture**, with additional middleware and service layers.





9. Running the Application

Running the Backend

The AI FAQ Assistant is a Node.js and Express.js backend application. The project is executed through the integrated terminal in Visual Studio Code.

Step 1: Open the Project

Open the **AI-FAQ-Assistant** project folder in Visual Studio Code.

Step 2: Open Terminal

In VS Code, select:

Terminal → New Terminal

Make sure the terminal is opened inside the project root folder.

Step 3: Install Dependencies

Run:

```
npm install
```

4. Run the Application

- **Development Mode** (auto-reloads on changes using native Node.js watch flag):

```
npm run dev
```

- **Production Mode:**

```
npm start
```

10. Environment Configuration

Create a `.env` file in the root directory.

`PORT=5000`

`MONGO_URI=mongodb://127.0.0.1:27017/ai_faq_assistant`

`JWT_SECRET=your_secure_jwt_secret`

`GEMINI_API_KEY=your_gemini_api_key`

Important: The `.env` file should not be uploaded to GitHub because it contains private credentials.

Instead, document the required variables without exposing the actual API key.

Create a `.env` file in the root directory (or use the provided template):

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/ai_faq_assistant
JWT_SECRET=your_jwt_secret_key_here_must_be_long_and_secure
GEMINI_API_KEY=your_google_gemini_api_key_here
```

`.env` file screenshot — hide/blur the actual Gemini API key and JWT secret.

11. Backend Development

Server Configuration

The `server.js` file starts the application, connects to MongoDB, and starts the Express server.

`src/server.js`

Express Application

The `app.js` file configures Express, middleware, routes, and error handling.

`src/app.js`

12. API Testing

The APIs can be tested using Thunder Client or Postman.

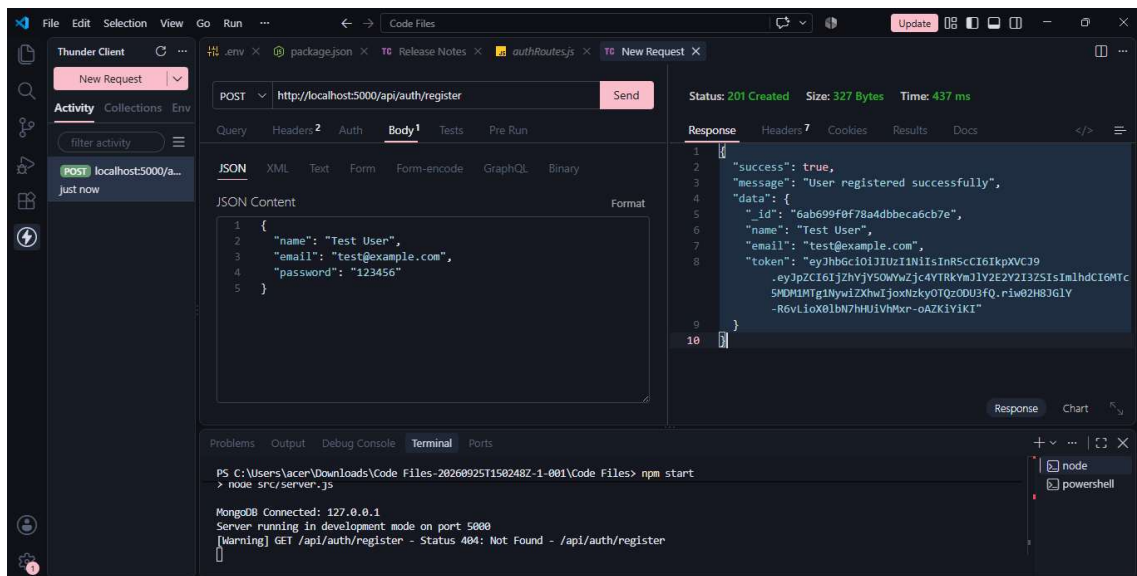
12.1 User Registration

Method:

POST /api/auth/register

Request Body:

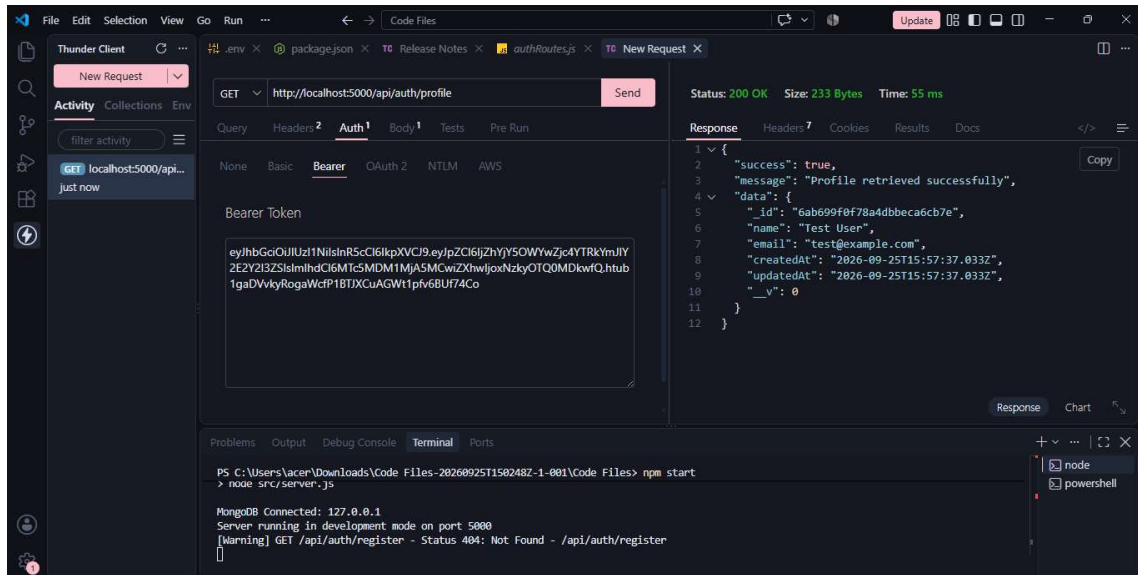
```
{  
  
  "name": "Test User",  
  
  "email": "test@example.com",  
  
  "password": "123456"  
}
```



REGISTER API — successful registration response.

12.2 User Login

This endpoint returns the authenticated user's profile information.



PROFILE API — successful profile response.

13. FAQ Operations

Create FAQ

Method:

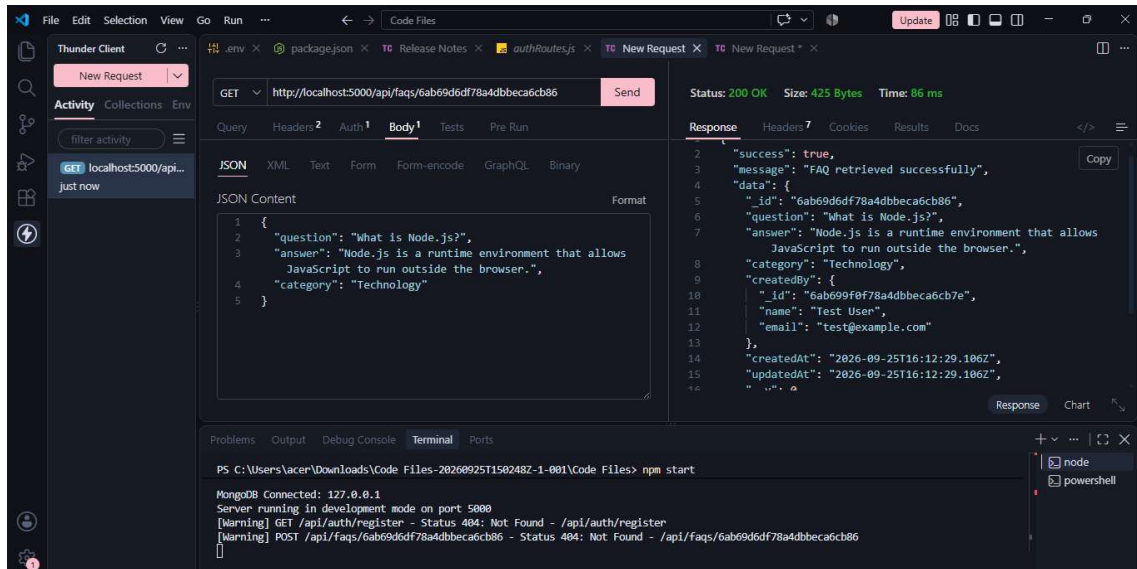
POST /api/faqs

Authorization:

Bearer <JWT_TOKEN>

Request Body:

```
{
  "question": "What is Node.js?",
  "answer": "Node.js is a JavaScript runtime environment.",
  "category": "Technology"
}
```



CREATE / POST FAQ — successful response.

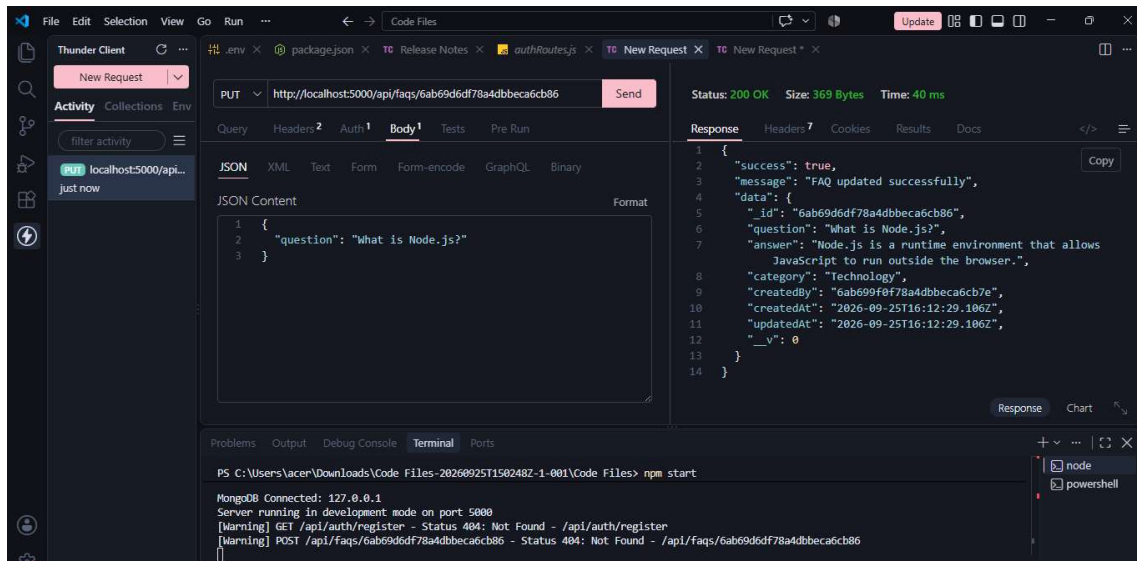
Update FAQ

Method:

PUT /api/faqs/:id

Authorization:

Bearer <JWT_TOKEN>



UPDATE / PUT FAQ — successful updated response.

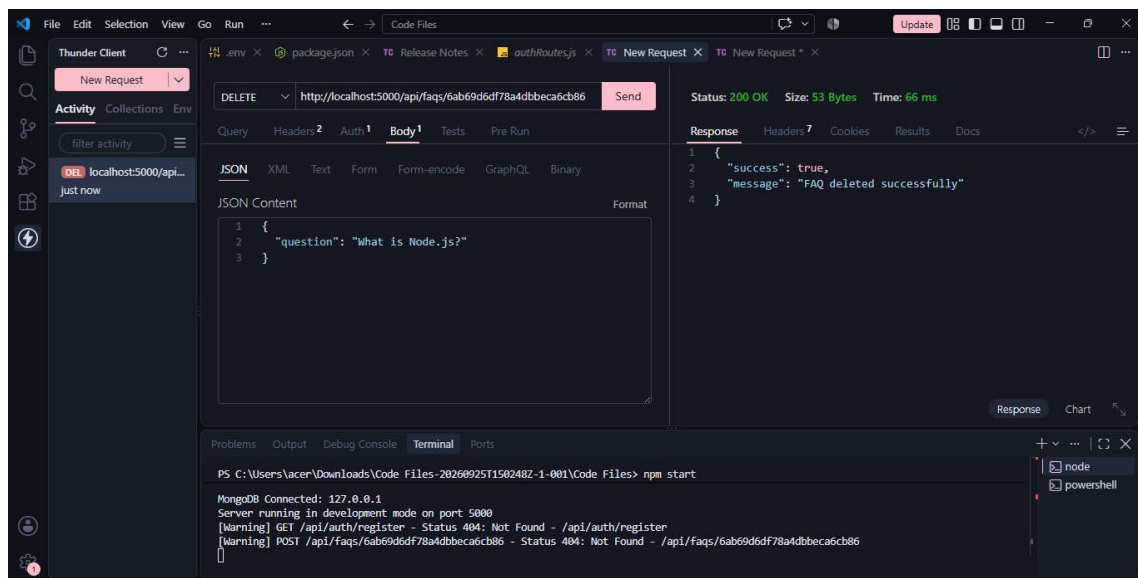
Delete FAQ

Method:

DELETE /api/faqs/:id

Authorization:

Bearer <JWT_TOKEN>



DELETE FAQ — successful deletion response.

AI FAQ Generation

Method:

POST /api/ai/generate-faq

Authorization:

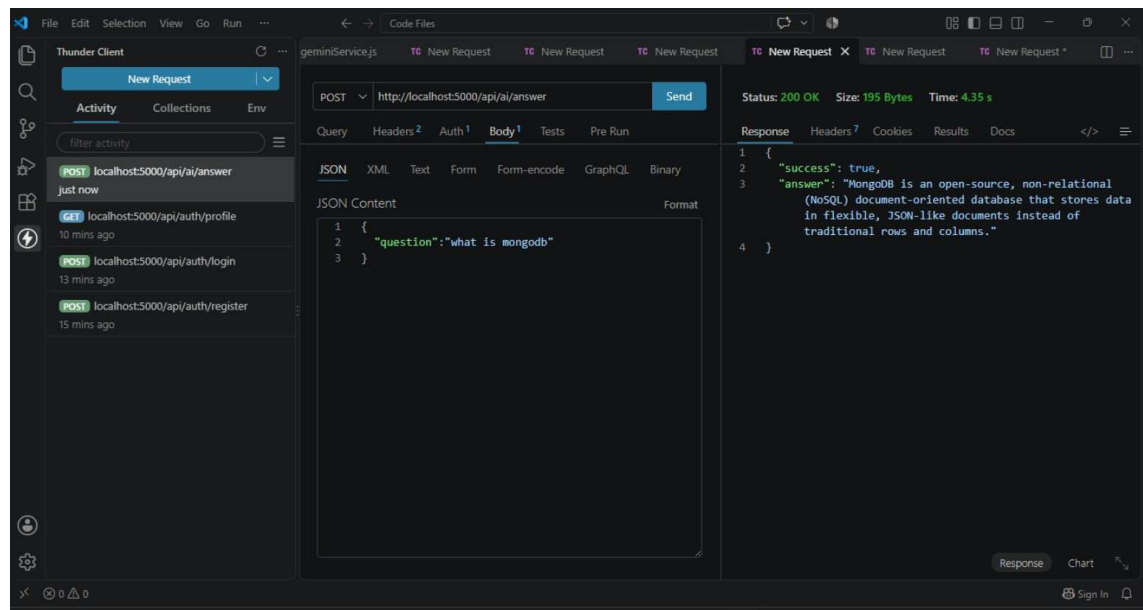
Bearer <JWT_TOKEN>

Request Body:

```
{  
  
  "topic": "MongoDB"
```

}

The Gemini service generates a structured FAQ containing a question and answer.



AI QUESTION/FAQ GENERATION — generated question and answer response.

14. API Summary

Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Register user
POST	/api/auth/login	Public	Login user
GET	/api/auth/profile	Protected	Get profile
POST	/api/faqs	Protected	Create FAQ

GET	<code>/api/faqs</code>	Public	Get FAQs
GET	<code>/api/faqs/:id</code>	Public	Get FAQ by ID
PUT	<code>/api/faqs/:id</code>	Protected	Update FAQ
DELETE	<code>/api/faqs/:id</code>	Protected	Delete FAQ
GET	<code>/api/faqs/search?q=</code>	Public	Search FAQs
POST	<code>/api/ai/answer</code>	Protected	Generate AI answer
POST	<code>/api/ai/generate-faq</code>	Protected	Generate AI FAQ

15. Running the Application

Start MongoDB and run the backend from the project directory.

npm start

The server runs on:

http://localhost:5000

The API can then be tested using Postman or Thunder Client.

16. Security

The project implements several security mechanisms:

- JWT authentication for protected routes.
 - bcrypt password hashing.
 - Environment variables for sensitive configuration.
 - Authentication middleware for private endpoints.
 - Request validation.
 - Centralized error handling.
 - Owner verification for FAQ modification and deletion.
 - `.env` excluded from GitHub using `.gitignore`.
-

17. Conclusion

The AI FAQ Assistant API provides a secure and modular backend solution for FAQ management and AI-powered customer support. The integration of Express.js, MongoDB, JWT authentication, bcrypt, and Google Gemini AI enables the system to manage user accounts, FAQ records, searches, and automated AI-generated responses.

The project demonstrates practical implementation of REST API development, authentication, database management, middleware, MVC architecture, and AI API integration.